



Tablouri unidimensionale

Probleme cu secvențe

Andrei Mihai Ciobanu

Algolymp Summer School

Introducere

Diverse abordări

Resurse

Probleme de antrenament

Introdurre

Definiție

Definiție

Se numește **secvență** a vectorului X o succesiune de **elemente consecutive** din X , în ordinea din X . Orice secvență a unui vector este unic determinată de doi indici $l \leq r$, ai primului, respectiv ultimului element din secvență.

Lungimea unei secvențe

Secvența $[l, r]$ are lungimea $r - l + 1$.

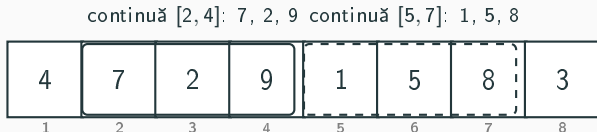
Exemplu: $[3, 7]$ are lungimea $7 - 3 + 1 = 5$.

Definiție alternativă

O secvență continuă se obține eliminând câteva elemente de la **începutul** șirului (posibil niciunul) și câteva de la **final** (posibil niciunul).

▷ Termenii «*continuă*» și «*contiguă*» se folosesc interschimbabil în contextul secvențelor.

Exemple de secvențe continue



Secvențe continue

- [2, 4]: valorile 7, 2, 9
- [5, 7]: valorile 1, 5, 8
- [4, 7]: valorile 9, 1, 5, 8
- [3, 3]: doar valoarea 2

Nu sunt continue

- {4, 2, 8}: poz. 1, 3, 7
- {7, 9, 5}: poz. 2, 4, 6

Câte secvențe există într-un șir?

Start	Capete posibile
1	1, 2, 3, 4, 5
2	2, 3, 4, 5
3	3, 4, 5
4	4, 5
5	5

Formula generală

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Problemă

La $n = 200.000$: circa 2×10^{10} secvențe. Avem nevoie de metode mai eficiente.

Varianta naivă — $O(n^3)$

```
1  int n, a[200001]; // ... citire ...
2  int lmax = 1, rmax = 0;
3  for (int l = 1; l <= n; l++) {
4      for (int r = l; r <= n; r++) {
5          bool ok = true;
6          for (int k = l; k <= r; k++)
7              if (a[k] % 2 == 0) { ok = false; break; }
8          if (ok && r - l + 1 > rmax - lmax + 1)
9              lmax = l, rmax = r;
10     }
11 }
12 // lungimea = rmax - lmax + 1
```

Complexitate: $O(n^3)$

Buclo interioară reface
verificarea complet.

Prea lentă!

La $n = 5.000$: $\approx 10^{10}$ operații.
 $\Rightarrow$ Mult prea încet.

Varianta de bază — $O(n^2)$

Eliminăm bucla interioară: parcurgem toate perechile (l, r) cu $l \leq r$, extinzând secvența cu câte un element:

```
1  int n, a[200001];
2  // ... citire n si a[1..n] ...
3
4  for (int l = 1; l <= n; l++) {
5      for (int r = l; r <= n; r++) {
6          // prelucreaza secventa a[l..r]
7      }
8  }
```

Când este suficientă?

Numai când $n \leq 3.000\text{--}5.000$. Dacă $n = 200.000$, avem nevoie de una din metodele din secțiunea următoare.

Diverse abordări

O metodă mai bună: proprietăți locale

Dacă știu că secvența $[l, r]$ este validă, pot afla rapid dacă $[l, r + 1]$ este validă? Dacă **da**: putem parcurge șirul o singură dată, nefiind nevoie de 2 bucle imbricate.

Exemple în care funcționează

- Secvențe strict crescătoare: extinderea depinde doar de $a[r]$ și $a[r + 1]$; orice subsecvență este de asemenea strict crescătoare
- Secvențe de numere prime: depinde doar dacă $a[r + 1]$ este prim; orice subsecvență respectă de asemenea această proprietate

Exemplu în care nu funcționează: $\gcd(a[l..r]) > 1$

Dacă $\gcd(a[l..r+1]) = 1$ (invalid), poate exista $l' > l$ cu $\gcd(a[l'..r+1]) > 1$. Nu știm unde să resetăm `len` — nu putem folosi simplu `len = 1`.

Secvența strict crescătoare — $O(n)$

Ținem variabila `len` = lungimea secvenței strict crescătoare care se termină **acum**. La fiecare pas: dacă $a[i - 1] < a[i]$, secvența continuă; altfel, reluăm de la 0.

```
1  int n, a[200001];
2  // ... citire ...
3
4  int len = 1, best = 1;
5  for (int i = 2; i <= n; i++) {
6      if (a[i-1] < a[i]) {
7          len++;                // secventa continua
8      } else {
9          len = 1;              // secventa s-a rupt, reluam de la
                                i
10     }
11     if (len > best) {
12         best = len;
13     }
14 }
15 cout << best << "\n";
```

Exemplu — șirul 1, 3, 5, 2, 3, 6, 7, 8, 4, 9

<i>i</i>	1	2	3	4	5	6	7	8	9	10
<i>a</i> [<i>i</i>]	1	3	5	2	3	6	7	8	4	9
len	1	2	3	1	2	3	4	5	1	2
best	1	2	3	3	3	3	4	5	5	5

Concluzii

Pozițiile 4–8: valorile 2, 3, 6, 7, 8 formează cel mai lung segment.

Răspuns: 5.

Problemă: Longest Increasing Contiguous Segment

Determină lungimea maximă a secvenței strict crescătoare continue. $1 \leq n \leq 200.000$.

Numărarea segmentelor cu o proprietate

Observație

Dacă secvența cu proprietatea P curentă are lungimea len , atunci exact len astfel de secvențe se termină la i .

Exemplu: șirul 2 4 7, proprietatea “strict crescător”

Secvențele strict crescătoare ce se termină la 7: $[7]$, $[4, 7]$, $[2, 4, 7]$
— total $3 = len$.

```
1 long long answer = 0;
2 int len = 1;
3 for (int i = 1; i <= n; i++) {
4     if (i > 1 && a[i-1] < a[i]) {
5         len++;
6     }
7     else {
8         len = 1;
9     }
10    answer += len;
11 }
12 cout << answer << "\n";
```

▷ Numărul de secvențe poate fi circa $n^2/2$; folosește `long long` pentru `answer`

Secvența de sumă maximă

Problemă

Găsește secvența cu suma maximă (elementele pot fi negative).

Algoritmul lui Kadane

Fie current = suma maximă a secvenței care se termină la poziția i .

Dacă $\text{current} < 0$: orice prelungire a prefixului curent e mai slabă decât una care începe proaspăt. Resetăm $\text{current} = 0$.

Altfel: extindem cu $a[i]$.

Complexitate: $O(n)$, memorie $O(1)$

O singură parcurgere; variabilele current și best .

Secvența de sumă maximă — implementare

```
1  int n, a[200001];
2  // ... citire ...
3  int best = a[1], current = a[1];
4  for (int i = 2; i <= n; i++) {
5      if (current < 0) {
6          current = 0;
7      }
8      current += a[i];
9      if (current > best) {
10         best = current;
11     }
12 }
13 cout << best << "\n";
```

Inițializarea `best = a[1]` (nu 0) tratează corect cazul în care toate elementele sunt negative.

Sliding Window

Problemă

Calculează suma fiecărei secvențe de lungime exact k . Variantă directă: $O(n \cdot k)$. Putem face $O(n)$.

Când fereastra se mută cu un pas la dreapta, iese $a[\text{left} - 1]$ și intră $a[\text{left} + k - 1]$:

```
1 long long sum = 0;
2 for (int i = 1; i <= k; i++) sum += a[i]; // prima fereastră
   [1, k]
3 // prelucrează fereastră [1, k]
4
5 for (int left = 2; left + k - 1 <= n; left++) {
6     sum -= a[left - 1]; // iese din stanga
7     sum += a[left + k - 1]; // intra din dreapta
8     // prelucrează fereastră [left, left+k-1]
9 }
```

Complexitate: $O(n)$

Sume parțiale: câte secvențe de lungime K au suma $\leq X$?

Idee

Suma secvenței $[l, l+K-1] = \text{pref}[l+K-1] - \text{pref}[l-1]$.
Iterăm l de la 1 la $n-K+1$ și numărăm câte satisfac condiția.

```
1 // pref[0] = 0, pref[i] = pref[i - 1] + a[i]
2 int answer = 0;
3 for (int l = 1; l + k - 1 <= n; l++) {
4     if (pref[l + k - 1] - pref[l - 1] <= x) {
5         answer++;
6     }
7 }
8 cout << answer << "\n";
```

Complexitate: $O(n)$ (precalculare + numărare)

Căutare binară pe sume parțiale

1. Împărțire în 2 secvențe de sume egale — $O(n)$

Fie $S = \text{pref}[n]$. Tăietura la i este validă dacă $\text{pref}[i] = S/2$.

Iterăm $i \in [1, n-1]$ și numărăm.

2. Împărțire în 3 secvențe de sume egale — $O(n \log n)$

Fixăm prima tăietură la i : $\text{pref}[i] = S/3$. Căutăm binar a doua tăietură $j \in [i+1, n-1]$ cu $\text{pref}[j] = 2S/3$.

(Necesită $a[i] \geq 0$, altfel sumele parțiale nu sunt crescătoare.)

3. Împărțire în 4 secvențe

Hint: cum putem adapta ideea de la problema 2?

Împărțire în 3 secvențe de sume egale — implementare

```
1 // S = pref[n], necesita S % 3 == 0 si a[i] >= 0
2 int answer = 0;
3 for (int i = 1; i <= n - 2; i++) {
4     if (3 * pref[i] != S) {
5         continue;
6     }
7     // ... cautam binar j ...
8     if (pref[j] == 2 * S / 3) {
9         answer++;
10    }
11 }
12 cout << answer << "\n";
```

Complexitate: $O(n \log n)$; atenție cand valorile pot fi nule $\Rightarrow j$ poate avea mai multe valori valide

Generalizare: K secvențe $\Rightarrow K-1$ tăieturi, căutăm binar fiecare $\Rightarrow O(K \log n)$ (doar pentru valori pozitive).

Pattern matching

Problemă: Pattern Inside a Sequence

Se dau şirul a (lungime n) şi şablonul b (lungime m). Numără câte poziţii de start i au proprietatea că $a[i..i+m-1] = b[1..m]$.

Restricţii: $1 \leq m \leq n \leq 3.000$. **Exemplu:** $a=[1,2,1,2,1]$, $b=[1,2,1] \Rightarrow$ răspuns 2.

```
1 int n, m, a[3001], b[3001], answer = 0;
2 for (int start = 1; start + m - 1 <= n; start++) {
3     bool ok = true;
4     for (int j = 1; j <= m; j++) {
5         if (a[start + j - 1] != b[j]) {
6             ok = false;
7             break;
8         }
9     }
10    if (ok) {
11        answer++;
12    }
13 }
14 cout << answer << "\n";
```

Complexitate: $O(n \cdot m)$, acceptabilă pentru $n, m \leq 3.000$.

Când Sliding Window nu este suficient

Sliding Window funcționează pentru lungime **fixă** k . Two Pointers = fereastră cu lungime variabilă.

Problemă: secvența de lungime maximă cu suma $\leq K$

Ținem doi indici $l \leq r$. Extindem r la dreapta; dacă suma depășește K , avansăm l . Fiecare index se mișcă cel mult n pași: $O(n)$ total.

Condiție de aplicabilitate

Proprietatea trebuie să fie **monotonă**: dacă $[l, r]$ e validă, atunci $[l, r-1]$ e de asemenea validă.

Resurse

1. **pbinfo** — Secvențe în vectori — definiție, complexități $O(n^3)/O(n^2)/O(n)$, exemple C++
2. **pbinfo** — Secvența de sumă maximă — algoritmul lui Kadane explicat pas cu pas

Probleme de antrenament

Probleme de antrenament

1. **Longest Increasing Contiguous Segment**
2. **Pattern Inside a Sequence**
3. **seism** (OJI 2021 VI)
4. **mandatar** (OJI 2025 VI)
5. **Avid** (OJI 2024 VI)
6. **zudt** (OJI 2026 VI)